

Distinct predictors of indexing failure and mapping accuracy in a human pangenome graph

Kazumichi Fujiwara¹ and Yosuke Kawai¹

¹ National Institute of Genetics

BioHackathon series:

DBCLS BioHackathon 2026

Matsuyama, Japan, 2026

BH26

Submitted: 18 Sep 2026

License:

Authors retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC-BY](#)).

Published by [BioHackrXiv.org](#)

Introduction

A pangenome graph represents many haplotypes at once, rather than forcing every sample onto one linear reference. The PanGenome Graph Builder (PGGB) builds such graphs without designating any input as privileged, aligning all haplotypes to one another and constructing the graph directly from that all-to-all alignment (Garrison et al., 2024), and it is one of the methods used by the Human Pangenome Reference Consortium (HPRC) to assemble a population-scale pangenome from long-read haplotype assemblies (Liao et al., 2023; Lucas et al., 2026). `vg giraffe` was introduced as a short-read mapper for graphs of this kind, seeding against a curated set of haplotype paths rather than the whole graph, which is what makes mapping to a graph with hundreds of haplotypes tractable at all (Sirén et al., 2021); long-read mapping was added subsequently (Chang et al., 2025).

Whether that combination scales past a single locus is still an open question. Chang and colleagues, describing the current state of Giraffe, report that they “have not been able to construct indexes for Giraffe” for PGGB graphs at genome scale, and attribute this directly to PGGB’s reference-free topology (Chang et al., 2025). That is a statement about outcome, not mechanism: it does not say what fraction of a genome would be affected, whether difficulty is concentrated in particular kinds of sequence, or whether topological difficulty and mapping inaccuracy are the same failure mode or two different ones.

Existing work on pangenome region difficulty answers adjacent questions. Li defines sample-agnostic “easy” regions for short-read variant calling by k-mer uniqueness across hundreds of assemblies, entirely independent of any graph (Li, 2025). Andreace and colleagues, and Dubois and colleagues, compare graphs built from the same genomes by different construction methods, and both find that the resulting structural disagreement concentrates in tandem repeats and other low-complexity sequence (Andreace et al., 2023; Dubois et al., 2025). None of these studies asks whether a single graph’s own local topology predicts what happens when that graph is indexed and mapped against.

We tested this directly on chromosome 21 of the HPRC release 2 PGGB graph, at a scale small enough to screen exhaustively: every 100 kb window on the chromosome. For each window we computed an explicit topology-based complexity score, and separately annotated known problem sequence classes (segmental duplications, large tandem repeats) from public UCSC tracks. We then asked which of the two, if either, predicts whether `vg autoindex` and `vg giraffe/vg surject` complete for a region, and which predicts the accuracy of the resulting short-read alignments once they do.

Methods

Graph and reference data

We used the HPRC release 2 PGGB pangenome graph for chromosome 21 (parameters p98-k311), which contains GRCh38 and CHM13 as reference paths alongside 464 HPRC year-2 haplotypes (466 haplotype/reference paths in total) (Garrison et al., 2024; Lucas et al., 2026). HPRC release 2 is roughly a fivefold expansion, in genome number, over the original draft pangenome (Liao et al., 2023; Lucas et al., 2026). Chromosome 21 was chosen as the smallest human autosome, to keep chromosome-scale extraction and screening tractable on a single workstation. An independent GRCh38 chromosome 21 FASTA (UCSC hg38) was used for N-content screening and as a graph-independent coordinate reference. All graph operations (build, extract, stats, degree, viz) used the odgi toolkit (Guarracino et al., 2022). Full provenance, including checksums and download dates for every input file, is recorded alongside this repository.

Candidate window screening

Every non-overlapping 100 kb window on chromosome 21 was considered, after excluding the first and last 2% of the chromosome and any window with more than 5% N content in the GRCh38 reference, leaving 367 candidate windows. Each was extracted from the graph with `odgi extract` and summarized with `odgi stats` and `odgi degree`. Windows were dropped from further analysis under two criteria. First, a degenerate-window filter (`edge_count == 0`, or fewer than 400 raw path fragments, or an extracted size more than three times the requested window) removed 22 windows clustered in the pericentromeric and acrocentric part of the chromosome, leaving 345. Second, because the raw path-fragment count used above does not distinguish a haplotype that is finely fragmented from a haplotype that is simply absent, we additionally counted the number of *distinct* haplotypes actually threading each window (parsed from the PanSN `sample#hap` path-name prefix via `odgi paths -L`). Nineteen windows, all within a contiguous stretch immediately flanking the centromere, had fewer than 450 of the 466 possible haplotypes represented and were removed, leaving 326 windows that were carried forward for scoring and sampling.

Complexity score

For each of the 326 valid windows we computed four graph-topology statistics from `odgi stats/odgi degree`: node density (nodes per kilobase of reference sequence), the edge-to-node ratio, sequence inflation (total graph sequence relative to the reference interval), and maximum node degree. Each statistic was log10-transformed and z-scored across the 326-window population, and the four z-scores were averaged into a single, unitless local complexity score. The score is a purely topological summary: it carries no information about sequence identity or content, which is what distinguishes it from the hazard annotation described next.

Known-hazard annotation

Complexity score alone cannot see sequence similarity to other parts of the genome, since it is computed from one local subgraph. To capture that separately, every valid window was checked against two UCSC tracks for chromosome 21, `genomicSuperDups` (Bailey et al., 2001) and `simpleRepeat` (Tandem Repeats Finder, (Benson, 1999)), both retrieved once for the whole chromosome and intersected locally rather than queried per window (Kent et al., 2002). A binary hazard flag was set for a window if it overlapped a segmental duplication at 90% identity or higher, or a tandem-repeat array spanning 1.4 kb or more; these thresholds were chosen because they correctly flagged every problem window identified during preliminary, hand-picked testing of this pipeline: two segmental duplications (one inter-chromosomal, one intra-chromosomal) and two large tandem-repeat or NUMT-containing windows. Under this

definition, 77 of the 326 windows (23.6%) carry a hazard flag. Crossing the hazard flag with a complexity tier (above or below the population median complexity score) gives a 2x2 design with four strata.

Region sampling and pipeline

Fifty windows were rank-sampled within each of the four complexity-tier-by-hazard-status strata, 25 per tier and 25 per hazard status by construction, so that rare combinations such as a low-complexity window carrying a known hazard would not be missed by chance. Each of the 50 regions was carried through an identical pipeline. Sequence graphs were extracted with `odgi extract` at the default subpath-merging distance; a minority of extractions that expanded past three times the requested window size, a known behaviour of the default merge distance next to large collapsed repeats, were re-extracted with merging disabled. Giraffe indexes were built with `vg autoindex --workflow sr-giraffe` (Sirén et al., 2021). For each region we simulated 30,000 error-free paired-end 150 bp reads (fragment length 350 bp $\pm$ 35 bp) from the GRCh38 interval with `wgsim` (Li, 2011), with a fixed seed for reproducibility. Reads were mapped with `vg giraffe` and surjected onto the linear GRCh38 path with `vg surject` (Sirén et al., 2021); BAM records were inspected with `samtools` (Danecek et al., 2021). A mapped read was scored correct if its absolute position, converted back from the region's local subpath coordinate, fell within 10 bp of either candidate fragment-boundary position implied by the simulated fragment (the read simulator does not record simulated strand, so either boundary is an acceptable match). Every pipeline stage for a region — extraction, indexing, simulation, mapping, surjection, evaluation — was run under a 30-minute wall-clock budget on a single machine (Apple Silicon, 16 cores, 128 GB memory); a region that had not produced a result within that budget was recorded as a pipeline failure. Two genes flagged as biologically notable in this analysis (the ribosomal RNA gene *RNA5-8SN3* and the *MIR6724* microRNA cluster overlapping the single most complex of the 345 scored windows, which was later excluded from the final 326-window pool by the haplotype- representativeness filter) were identified by looking up their coordinates against Ensembl gene annotation (Dyer et al., 2025). All downstream tabulation and the figures in this report were produced with `pandas` (McKinney, 2010) and `matplotlib` (Hunter, 2007).

Results

Forty-six of the 50 sampled regions completed indexing, mapping and surjection within the 30-minute budget; four did not. All four failures fell in the above-median complexity tier, giving a failure rate of 16% (4/25, 95% exact binomial CI 5-36%) in the high-complexity tier against 0% (0/25, CI 0-14%) in the low tier. Hazard status separates this same outcome far less sharply: 12% (3/25, CI 3-31%) of hazard-flagged regions failed against 4% (1/25, CI 0-20%) of regions with no known hazard, intervals that overlap almost entirely. Complexity tier, not hazard status, is what distinguishes regions that finish from regions that do not.

Mapping accuracy among the 46 completed regions inverts this pattern. Complexity tier separates it poorly: minimum accuracy was 98.7% in the low tier and 99.3% in the high tier, so the worst-mapping region is not the most complex one. Hazard status separates it cleanly: mean accuracy was 99.92% with no hazard flag against 99.84% with one, and minimum accuracy 99.53% against 98.69%. The single worst-performing region in the entire sample — 98.69% accuracy — is in fact the single *lowest*-complexity region in the whole 326-window population (complexity score -2.17), and it carries a hazard flag: `genomicSuperDups` lists 41 segmental-duplication entries overlapping this window at 90-97% identity, matching paralogous sequence on six other chromosomes. A complexity score computed from one local subgraph has no way to detect that the same sequence is duplicated elsewhere in the genome, and ranks this window as the simplest one tested.

The complexity score distribution across the 326-window population is strongly right-skewed

(Fig. 1): 77.6% of windows fall within half a standard deviation of the median, with a long tail of rare extremes. A sample drawn uniformly across this distribution, or even rank-sampled across its full range without regard to hazard status, would rarely include the rare, extreme, or hazard-flagged windows that carry the outcomes reported above — which is why we sampled within complexity-tier-by-hazard strata rather than across the pool as a whole.

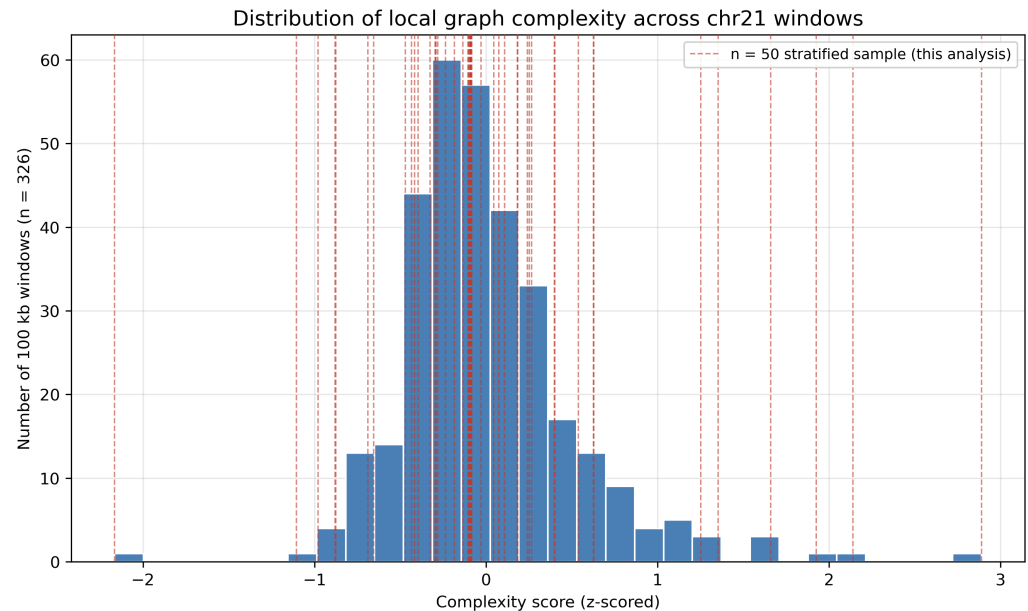


Figure 1: Distribution of local complexity score across the 326 valid 100 kb windows on chromosome 21, with the 50 sampled regions marked by dashed lines

Figure 2 lays out the full result. Panel a plots complexity score against accuracy for all 50 regions, colored by hazard status, with the four incomplete regions plotted at the foot of the axis rather than omitted; the lowest-accuracy points are disproportionately hazard-flagged (red), while the four failures (black triangles) cluster toward high complexity regardless of hazard status. Panel b isolates the failure-rate contrast by complexity tier; panel c isolates the accuracy contrast by hazard status. No single panel makes the case on its own — together they show that the two axes of difficulty are orthogonal.

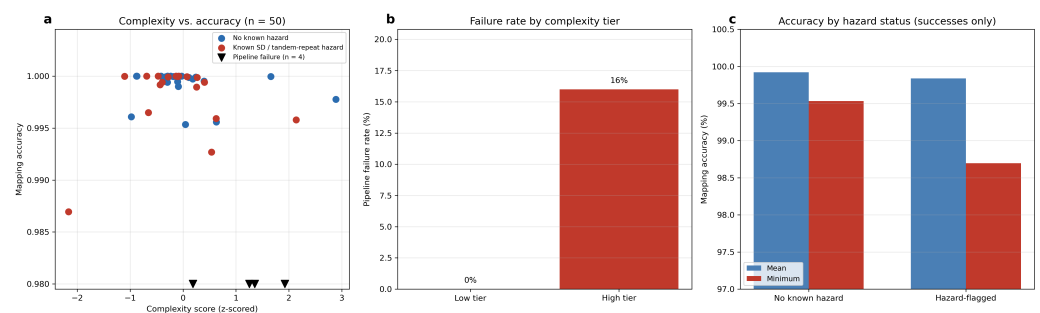


Figure 2: Complexity score, pipeline completion, and mapping accuracy across the 50 sampled regions. (a) complexity score versus accuracy, colored by known-hazard status, with pipeline failures marked at the bottom of the axis; (b) pipeline failure rate by complexity tier; (c) mean and minimum accuracy among completed regions, by hazard status

Two considerations qualify how far these numbers should be pushed. First, “failed to complete”

means no result within a fixed 30-minute, single-machine budget, not proof that a region cannot be indexed at all. Two regions encountered during preliminary testing of this pipeline were initially killed as failures under a shorter 10-minute budget and later completed successfully once given 30 minutes; one of them is part of this sample — the single highest-complexity region tested (score 2.89) — and it completed here at 99.78% accuracy after roughly 26 minutes of total wall-clock time, almost all of it spent in mapping and surjection rather than indexing (index construction itself took 36 seconds, the value reported in the per-region results table). The 16% failure rate we report for the high-complexity tier is therefore an upper bound under this specific compute budget, not a fixed property of the graph. Second, that figure and its counterparts rest on four, three, and one events out of 25 regions, respectively; the 95% confidence intervals above are correspondingly wide, and the *direction* of each effect should be trusted well before its exact magnitude.

Discussion

Local graph topology and known sequence hazard are not two measures of the same thing; they govern different stages of the pipeline. Complexity score tracks whether indexing and mapping complete at all, and is blind to accuracy among the regions that do complete. Hazard status tracks accuracy among completed regions, and is blind to which regions fail outright. A screen built on either measure alone would miss exactly the risk the other one catches — a complexity-only screen would have called our worst-mapping region the safest one in the sample.

Chang and colleagues report that Giraffe indexing has not been made to work at genome scale for PGGB graphs (Chang et al., 2025); we do not resolve that problem, but our results sharpen what it looks like below genome scale. Tractability failure at 100 kb is not confined to the single most extreme window on the chromosome: it is a property of the upper half of the complexity distribution generally, affecting a double-digit percentage of windows tested under an ordinary compute budget. A genome-wide extension of Giraffe indexing will need to treat outright failure as an expected outcome to be measured and reported, not an edge case to be debugged away.

Two consequences follow directly for anyone screening candidate regions at scale. Existing, cheaply computed sequence annotation — segmental duplication and tandem-repeat tracks that already exist for the human genome — should be checked alongside any topology-based complexity score, because the two are not substitutes and a complexity ranking alone will misclassify a subset of the worst regions as the safest. And because at least two regions in this study that looked like outright failures under a short timeout turned out simply to be slow, whatever timeout a larger screen adopts should be generous enough that “failed” means something more than “ran out of patience.”

Most of the limitations below trace back to a single cause: this work was carried out during one BioHackathon, under a fixed and fairly short window of both time and compute, and that constraint shaped the scope of the study as much as any scientific choice did. All extraction, indexing, and mapping ran on one workstation rather than a cluster or cloud allocation, because no other compute was available during the event. The 30-minute per-region timeout was set to let the full 50-region pipeline finish inside the time remaining in the hackathon, not derived from any property of the graph itself; a more patient budget, or more machines run in parallel, would plausibly change the specific failure counts reported above, as the two regions discussed earlier already demonstrate. Screening was limited to chromosome 21, chosen as the smallest human autosome specifically so that exhaustive 100 kb screening would finish in the available time, so we did not test whether the failure and hazard rates reported here hold on larger, more repeat-rich chromosomes with a different balance of segmental duplication, tandem repeat, and pericentromeric content. Reads were simulated error-free from the linear reference rather than drawn from a real sequencing run, again because acquiring and processing real short-read data at this scale was not feasible inside the hackathon’s time budget; this reports graph

and mapper behaviour under idealized input, not real Illumina sequencing performance. And the sample of 50 regions, while stratified to guarantee coverage of rare hazard-by-complexity combinations, has a size set by how many regions could be carried through the full pipeline before time ran out, not by a power calculation — the 95% confidence intervals in the Results section are the direct, quantitative consequence of that constraint.

One further limitation is not a matter of time or compute. A truth model built from a single simulated coordinate per read cannot distinguish a genuine mapping error from a read placed correctly at one of several biologically valid locations in a duplicated region — precisely the situation a hazard-flagged window is most likely to create — which would bias our accuracy numbers downward rather than up. Real sequencing reads, additional and larger chromosomes, a longer or resource-scaled timeout, and a repeat-aware definition of mapping correctness would each remove one of the constraints above; none of them calls for a different pipeline, only for more time and compute than a single hackathon provides.

Acknowledgements

We thank the organizers and participants of BioHackathon 2026 for the time and computing environment that made this analysis possible, and the HPRC, PGGB, and vg development teams for making the underlying graphs and tools openly available.

References

Author contributions

Kazumichi Fujiwara: Conceptualization, Investigation, Software, Writing – original draft; Yosuke Kawai: Writing – review & editing.

References

- Andreace, F., Lechat, P., Dufresne, Y., & Chikhi, R. (2023). Comparing methods for constructing and representing human pangenome graphs. *Genome Biology*, 24(1), 274. <https://doi.org/10.1186/s13059-023-03098-2>
- Bailey, J. A., Yavor, A. M., Massa, H. F., Trask, B. J., & Eichler, E. E. (2001). Segmental duplications: organization and impact within the current human genome project assembly. *Genome Research*, 11(6), 1005–1017. <https://doi.org/10.1101/gr.187101>
- Benson, G. (1999). Tandem repeats finder: a program to analyze DNA sequences. *Nucleic Acids Research*, 27(2), 573–580. <https://doi.org/10.1093/nar/27.2.573>
- Chang, X., Novak, A. M., Eizenga, J. M., Sirén, J., Monlong, J., Negi, S., Andreace, F., Nag, S., Kyriakidis, K., Hickey, G., Hwang, S., Délot, E. C., Carroll, A., Shafin, K., Chang, P.-C., Okamoto, F., & Paten, B. (2025). Rapid, accurate long- and short-read mapping to large pangenome graphs with vg Giraffe. *bioRxiv*. <https://doi.org/10.1101/2025.09.29.678807>
- Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giab008. <https://doi.org/10.1093/gigascience/giab008>
- Dubois, S., Zytnicki, M., Lemaitre, C., & Faraut, T. (2025). Pairwise graph edit distance characterizes the impact of the construction method on pangenome graphs. *Bioinformatics*, 41(6), btaf291. <https://doi.org/10.1093/bioinformatics/btaf291>
- Dyer, S. C., Austine-Orimoloye, O., Azov, A. G., Barba, M., Barnes, I., et al. (2025). Ensembl 2025. *Nucleic Acids Research*, 53(D1), D948–D957. <https://doi.org/10.1093/nar/gkae1071>
- Garrison, E., Guarracino, A., Heumos, S., Villani, F., Bao, Z., Tattini, L., Hagmann, J., Vorbrugg, S., Marco-Sola, S., Kubica, C., Ashbrook, D. G., Hickey, G., Sirén, J., Eizenga, J. M., Consortium, H. P. R., Cartwright, R. A., Barrio, A. M., Ghurye, J., Abramson, A., et al. (2024). Building pangenome graphs. *Nature Methods*, 21, 2008–2012. <https://doi.org/10.1038/s41592-024-02430-3>
- Guarracino, A., Heumos, S., Nahnsen, S., Prins, P., & Garrison, E. (2022). ODGI: understanding pangenome graphs. *Bioinformatics*, 38(13), 3319–3326. <https://doi.org/10.1093/bioinformatics/btac308>

- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- Kent, W. J., Sugnet, C. W., Furey, T. S., Roskin, K. M., Pringle, T. H., Zahler, A. M., & Haussler, D. (2002). The human genome browser at UCSC. *Genome Research*, 12(6), 996–1006. <https://doi.org/10.1101/gr.229102>
- Li, H. (2011). wgsim: a small tool for simulating sequence reads from a reference genome.
- Li, H. (2025). Finding easy regions for short-read variant calling from pangenome data. *GigaScience*, 14, g1af103. <https://doi.org/10.1093/gigascience/g1af103>
- Liao, W.-W., Asri, M., Ebler, J., Doerr, D., Haukness, M., Hickey, G., Lu, S., Lucas, J. K., Monlong, J., Abel, H. J., et al. (2023). A draft human pangenome reference. *Nature*, 617(7960), 312–324. <https://doi.org/10.1038/s41586-023-05896-x>
- Lucas, J. K., Hebbar, P., Liao, W.-W., Macias-Velasco, J. F., Novak, A. M., Altemose, N., Garrison, E., Hall, I. M., Jarvis, E. D., Paten, B., et al. (2026). HPRC2: A human pangenome reference with near-complete coverage of common genetic variation. *bioRxiv*. <https://doi.org/10.64898/2026.07.21.739710>
- McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>
- Sirén, J., Monlong, J., Chang, X., Novak, A. M., Eizenga, J. M., Markello, C., Sibbesen, J. A., Hickey, G., Chang, P.-C., Carroll, A., Gupta, N., Gabriel, S., Blackwell, T. W., Ratan, A., Taylor, K. D., Rich, S. S., Rotter, J. I., Haussler, D., Garrison, E., & Paten, B. (2021). Pangenomics enables genotyping of known structural variants in 5202 diverse genomes. *Science*, 374(6574), abg8871. <https://doi.org/10.1126/science.abg8871>



Appendices

Full methodology detail, per-region result tables, and the narrative logs documenting the preliminary hand-picked testing referenced above are available in this repository's `README.md` and `logs/` directory.